

TOPIC 5

Arrays, Collections & String Handling

5.1 One-Dimensional Arrays — Declaration, Initialisation & Traversal

An array is a fixed-size, ordered collection of elements of the same type stored in contiguous memory. Once created, an array's length cannot change. Arrays are objects in Java; the variable holds a reference to the array.

Declaration Styles

```
// Style 1: declare then allocate (length fixed at runtime)
int[] scores;
scores = new int[5]; // indices 0-4, all initialised to 0

// Style 2: declare and allocate in one line
double[] prices = new double[10];

// Style 3: array initialiser literal (length inferred)
int[] primes = {2, 3, 5, 7, 11, 13};
String[] days = {"Mon", "Tue", "Wed", "Thu", "Fri"};

// Reading and writing elements (zero-based index)
scores[0] = 95;
scores[4] = 78;
int first = scores[0]; // 95
int len = scores.length; // 5 - .length is a field, not a method!
```

⚠ ArrayIndexOutOfBoundsException

Valid indices are 0 to array.length-1. Accessing index -1 or index == length throws ArrayIndexOutOfBoundsException at runtime. Always guard loops with `i < array.length`, never `i <= array.length`.

Default Values After Allocation

Type	Default Value
int, short, byte, long	0
float, double	0.0
char	'\u0000' (null character)
boolean	false
Object (String, etc.)	null

Array Traversal

```
int[] grades = {88, 92, 75, 61, 99, 84};

// Method 1: indexed for loop (use when you need the index)
```

```

for (int i = 0; i < grades.length; i++) {
    System.out.printf("grades[%d] = %d\n", i, grades[i]);
}

// Method 2: enhanced for-each (cleaner, read-only)
for (int g : grades) {
    System.out.println(g);
}

// Method 3: Arrays utility (quick print)
import java.util.Arrays;
System.out.println(Arrays.toString(grades));
// [88, 92, 75, 61, 99, 84]

```

5.2 Common Array Algorithms

Linear Search

Linear search scans each element from index 0 until the target is found or the array is exhausted. Time complexity: $O(n)$ — proportional to array size.

```

public static int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) return i; // found at index i
    }
    return -1; // convention: -1 means not found
}

// Usage:
int[] data = {14, 27, 3, 88, 45};
int idx = linearSearch(data, 88); // returns 3
int missing = linearSearch(data, 99); // returns -1

```

Min, Max, Sum & Average

```

public static void statistics(int[] arr) {
    int min = arr[0], max = arr[0], sum = 0;

    for (int val : arr) {
        sum += val;
        if (val < min) min = val;
        if (val > max) max = val;
    }

    double avg = (double) sum / arr.length;
    System.out.printf("Min=%d Max=%d Sum=%d Avg=%.2f\n", min, max, sum,
avg);
}

```

Sorting with Arrays.sort() & Manual Bubble Sort

```

// Built-in sort (uses Dual-Pivot Quicksort — very fast)
import java.util.Arrays;

```

```

int[] nums = {5, 2, 8, 1, 9, 3};
Arrays.sort(nums);
System.out.println(Arrays.toString(nums)); // [1, 2, 3, 5, 8, 9]

// Manual Bubble Sort — teaches the concept
public static void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                int tmp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = tmp;
            }
        }
    }
}

```

5.3 Passing Arrays to Methods

When you pass an array to a method, the reference is copied — both the caller and the method share the same array object in memory. Changes made inside the method are visible to the caller. This is fundamentally different from primitives.

```

public static void doubleAll(int[] arr) {
    for (int i = 0; i < arr.length; i++) {
        arr[i] *= 2; // modifies the ORIGINAL array
    }
}

// Returning a new array (non-destructive)
public static int[] copyAndDouble(int[] src) {
    int[] result = new int[src.length];
    for (int i = 0; i < src.length; i++) result[i] = src[i] * 2;
    return result;
}

// Arrays.copyOf — built-in copy
int[] original = {1, 2, 3, 4, 5};
int[] clone = Arrays.copyOf(original, original.length);
int[] partial = Arrays.copyOfRange(original, 1, 4); // {2,3,4}

```

5.4 Multi-Dimensional Arrays (2D)

A 2D array in Java is an array of arrays. Think of it as a table with rows and columns. Each row can have a different length (jagged arrays), though square/rectangular layouts are most common.

```

// Declaration and allocation: 3 rows × 4 columns
int[][] matrix = new int[3][4];

// Initialiser literal
int[][] grid = {

```

```

    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

// Access: grid[row][col]
int center = grid[1][1]; // 5
grid[0][2] = 99;         // top-right corner = 99

// Traversal: nested for loops
for (int row = 0; row < grid.length; row++) {
    for (int col = 0; col < grid[row].length; col++) {
        System.out.printf("%4d", grid[row][col]);
    }
    System.out.println();
}

// Useful dimensions:
int rows = grid.length; // 3
int cols = grid[0].length; // 3 (assumes rectangular)

```

Practical 2D Example — Student Score Matrix

```

// 4 students × 3 subjects
double[][] scores = {
    {85.0, 92.0, 78.0}, // Student 0
    {70.0, 88.0, 95.0}, // Student 1
    {60.0, 72.0, 80.0}, // Student 2
    {91.0, 85.0, 89.0}  // Student 3
};

// Compute average score per student
for (int i = 0; i < scores.length; i++) {
    double sum = 0;
    for (double s : scores[i]) sum += s;
    System.out.printf("Student %d avg: %.2f\n", i, sum / scores[i].length);
}

```

5.5 Introduction to ArrayList<T>

ArrayList is a resizable array from the java.util package. Unlike arrays, it grows and shrinks dynamically. It stores objects only — for primitives, use the corresponding wrapper type (Integer, Double, etc.).

Core Operations

```

import java.util.ArrayList;

// Create an ArrayList of Strings
ArrayList<String> names = new ArrayList<>(); // diamond inference

// add() — append to end
names.add("Alice");
names.add("Bob");
names.add("Charlie");

```

```

names.add(1, "Zara");    // insert at index 1 → [Alice, Zara, Bob, Charlie]

// get() — retrieve by index
String first = names.get(0);    // "Alice"

// set() — replace element
names.set(2, "Brian");          // replaces Bob with Brian

// remove() — by index or by value
names.remove(0);                // removes Alice (index 0)
names.remove("Charlie");        // removes first occurrence of
"Charlie"

// size() — number of elements
int count = names.size();

// contains() — membership check
boolean found = names.contains("Zara");    // true

// indexOf() — first occurrence index
int idx = names.indexOf("Zara");    // 0

// clear() — remove all elements
names.clear();

// Traversal
for (String name : names) System.out.println(name);

```

ArrayList vs Array — When to Use Which

Criterion	Array vs ArrayList
Size	Array: fixed at creation. ArrayList: dynamic — grows/shrinks automatically.
Type	Array: works with primitives AND objects. ArrayList: objects only (use Integer, Double, etc. for primitives).
Performance	Array: slightly faster for random access (no overhead). ArrayList: tiny overhead per operation.
Methods	Array: only .length and Arrays utility class. ArrayList: rich API — add, remove, contains, sort, subList, etc.
Use when	Array: size known at compile time, primitive types, performance-critical inner loops. ArrayList: size unknown, frequent insertions/removals.

5.6 The String Class

String is one of the most-used classes in Java. Strings are immutable — every 'modification' creates a new String object. The original is unchanged. String literals are pooled in the String Pool for memory efficiency.

Essential String Methods

```
String s = "Hello, World!";
```

```

// length() - number of characters
s.length();           // 13

// charAt(index) - character at position
s.charAt(0);           // 'H'
s.charAt(s.length() - 1); // '!'

// substring(start) or substring(start, end)
s.substring(7);         // "World!" (from index 7 to end)
s.substring(7, 12);     // "World" (indices 7-11, end exclusive)

// indexOf() - position of first occurrence
s.indexOf('o');         // 4
s.indexOf("World");     // 7
s.lastIndexOf('o');     // 8
s.indexOf("xyz");       // -1 (not found)

// toUpperCase() / toLowerCase()
s.toUpperCase();        // "HELLO, WORLD!"
s.toLowerCase();        // "hello, world!"

// trim() / strip() - remove leading & trailing whitespace
String padded = " hello ";
padded.trim();          // "hello" (legacy)
padded.strip();         // "hello" (Java 11+ - Unicode-aware)

// replace() - replace all occurrences
s.replace("World", "Java"); // "Hello, Java!"

// contains() - substring membership
s.contains("World");      // true

// startsWith() / endsWith()
s.startsWith("Hello");   // true
s.endsWith("!");         // true

// split() - divide into array of tokens
String[] words = s.split(", "); // ["Hello", "World!"]

// compareTo() - lexicographic comparison
int cmp = "apple".compareTo("banana");
// negative: apple < banana; 0: equal; positive: apple > banana

// CRITICAL: use equals() to compare content, NOT ==
String a = new String("Java"), b = new String("Java");
a == b; // false - compares references
a.equals(b); // true - compares content ✓
a.equalsIgnoreCase(b); // true - case-insensitive ✓

```

⚠ String Immutability — The Hidden Performance Trap

Every time you use `+` to concatenate inside a loop, Java creates a new `String` object each iteration. For n iterations this creates $O(n)$ throwaway objects. Inside loops, ALWAYS use `StringBuilder` instead.

5.7 StringBuilder — Mutable String Operations

StringBuilder provides a mutable sequence of characters. Unlike String, modifications are made in-place. Use it when building strings incrementally — especially inside loops.

```
StringBuilder sb = new StringBuilder();

// append() — add to end (accepts any type)
sb.append("Java");
sb.append(" is");
sb.append(" powerful");
sb.toString(); // "Java is powerful"

// insert(index, str)
sb.insert(5, "17 "); // "Java 17 is powerful"

// delete(start, end)
sb.delete(4, 7); // removes characters at indices 4,5,6

// replace(start, end, str)
sb.replace(0, 4, "Kotlin");

// reverse()
sb.reverse(); // reverses the entire content

// length() and charAt() work the same as String
sb.length();
sb.charAt(0);

// Performance comparison: build a 10,000-char string
// BAD — creates 10,000 intermediate String objects:
String slow = "";
for (int i = 0; i < 10000; i++) slow += "x"; // 🐢 slow

// GOOD — single object, all appends in-place:
StringBuilder fast = new StringBuilder();
for (int i = 0; i < 10000; i++) fast.append("x"); // 🚀 fast
String result = fast.toString();
```



Lab 5 — Student Grade Tracker & String Parsing



Exercise Set A: 1D Array Foundations

- Exercise 1. Write a program that reads 10 integers from the user into an array, then prints them in reverse order.
- Exercise 2. Write a method that accepts an int array and returns a new array containing only the even numbers from the original.
- Exercise 3. Write a method `int[] removeDuplicates(int[] arr)` that returns a new array with all duplicate values removed, preserving order.

- Exercise 4. Write a method that counts how many elements in an array are above the array's average. Print both the average and the count.
- Exercise 5. Implement a method that rotates an array left by k positions. Example: {1,2,3,4,5} rotated left by 2 becomes {3,4,5,1,2}.
- Exercise 6. Write a method that merges two sorted arrays into a single sorted array without using Arrays.sort().
- Exercise 7. Implement Selection Sort: find the minimum in the unsorted portion and swap it into position. Trace each pass visually for an array of 6 elements.

**Exercise Set B: Student Grade Tracker (Core Project)**

- Exercise 8. **Basic Tracker:** Create an array of 5 student names and a parallel array of their scores (0–100). Print a formatted report: Name | Score | Grade. Compute and print the class average.
- Exercise 9. **Dynamic Tracker with ArrayList:** Rewrite Exercise 8 using ArrayList<String> for names and ArrayList<Double> for scores. Allow the user to add, remove, or update students via a menu. Print the roster after each operation.
- Exercise 10. Add to Exercise 9: a method that returns the top-N students by score (user specifies N). Print them in descending rank order.
- Exercise 11. Add grade distribution reporting: count how many students got A, B, C, D, and F. Display as both counts and percentages.
- Exercise 12. **2D Grade Matrix:** Build a 5-student × 4-subject score matrix. Compute and print: each student's average, each subject's average, the highest and lowest scoring student per subject.
- Exercise 13. Add a method that normalises scores in the 2D matrix so that the highest score in each subject becomes 100 and others scale proportionally.

**Exercise Set C: String Parsing & Processing**

- Exercise 14. Write a method that counts the frequency of each vowel (a, e, i, o, u) in a given string. Display the result as a table.
- Exercise 15. Write a method that converts a sentence to Title Case (first letter of each word capitalised, rest lowercase). Do NOT use any library method for this — use charAt() and Character class.
- Exercise 16. Write a method boolean isAnagram(String a, String b) that returns true if a and b contain the same characters in any order (ignore spaces and case).
- Exercise 17. Write a method that validates a password: minimum 8 characters, at least one uppercase, one lowercase, one digit, and one special character. Return a descriptive message listing all failed criteria.
- Exercise 18. Write a CSV parser: given a line like "Smith, John",28,"New York", extract each field correctly (handling commas inside quoted fields).
- Exercise 19. Write a method String compressRLE(String s) using Run-Length Encoding. Example: 'aaabbbcca' → 'a3b3c2a1'. Also write the decompressor.
- Exercise 20. **StringBuilder Challenge:** Build a simple template engine. Given a template string like 'Dear {name}, your balance is {balance}.', and a map of key-value pairs, replace all {key} tokens with their values using StringBuilder.



Exercise Set D: Challenge Problems

Exercise 21. Implement a spell-checker prototype: read a dictionary of valid words into an `ArrayList`, then check each word in a user's sentence. Print 'Unknown: X' for words not in the dictionary.

Exercise 22. Write a method that finds the longest common substring of two strings.
Example: 'abcdef' and 'bcd fgh' → 'bcd'.

Exercise 23. Build a word frequency counter: read a paragraph of text, split into words (clean punctuation), count each word's occurrences, and print the top-10 most frequent words in descending order.

TOPIC 6

Object-Oriented Programming — Classes & Objects

6.1 Core OOP Concepts

Object-Oriented Programming is a paradigm that organises software around objects — self-contained units that combine data (state) with behaviour (operations). Java's OOP model rests on four foundational pillars:

Pillar	Core Idea	Java Mechanism
Encapsulation	Bundle data and methods together; hide implementation details from the outside world.	private fields + public getters/setters; access modifiers.
Inheritance	A new class acquires properties and behaviour of an existing class, enabling code reuse.	extends keyword; method overriding; super reference.
Polymorphism	The same operation behaves differently depending on the actual object type at runtime.	Method overriding + dynamic dispatch; interfaces; method overloading.
Abstraction	Expose only essential features; hide complexity. Define WHAT, not HOW.	abstract classes; interfaces; information hiding.



Why OOP?

Procedural programs (Topic 1–4) model a problem as a sequence of steps. OOP models it as a community of collaborating objects — each responsible for its own data and actions. This maps more naturally to real-world domains and scales to large, team-built systems.

6.2 Defining Classes — Fields & Methods

A class is a blueprint (template) for creating objects. It declares fields (what the object knows) and methods (what the object can do). An object is a specific instance of a class created at runtime with the new keyword.

Anatomy of a Class

```
// File: Car.java
public class Car {

    // ——— FIELDS (instance variables) ———
    private String make;           // manufacturer
    private String model;
    private int year;
    private double speed;          // current speed km/h
    private static int totalCars = 0; // shared by all instances

    // ——— CONSTRUCTOR ———
    public Car(String make, String model, int year) {
        this.make = make;
    }
}
```

```

        this.model = model;
        this.year = year;
        this.speed = 0;
        totalCars++;
    }

    // ——— METHODS (behaviour) ———
    public void accelerate(double amount) {
        if (amount > 0) speed += amount;
    }

    public void brake(double amount) {
        speed = Math.max(0, speed - amount);
    }

    public String getInfo() {
        return String.format("%d %s %s @ %.1f km/h", year, make, model,
speed);
    }

    public static int getTotalCars() { return totalCars; }

    // toString() - called automatically by println()
    @Override
    public String toString() { return getInfo(); }
}

```

Using the Car Class

```

public class Main {
    public static void main(String[] args) {
        Car c1 = new Car("Toyota", "Corolla", 2022);
        Car c2 = new Car("Ford", "Mustang", 2023);

        c1.accelerate(60);
        c1.accelerate(30);
        c1.brake(20);

        System.out.println(c1);                // 2022 Toyota Corolla @ 70.0
        km/h
        System.out.println(c2);                // 2023 Ford Mustang @ 0.0
        km/h
        System.out.println(Car.getTotalCars()); // 2
    }
}

```

6.3 Constructors — Default, Parameterised & Overloaded

A constructor is a special method called by the new keyword to initialise an object. It has the same name as the class and no return type. If you define no constructor, Java provides a default (no-arg) constructor automatically. As soon as you define any constructor, the automatic default disappears.

```

public class Student {

```

```

private String name;
private int age;
private double gpa;

// 1) Default (no-arg) constructor
public Student() {
    this("Unknown", 18, 0.0); // delegates to 3-arg constructor
}

// 2) Name-only constructor
public Student(String name) {
    this(name, 18, 0.0);
}

// 3) Full constructor — canonical form
public Student(String name, int age, double gpa) {
    this.name = name;
    this.age = age;
    this.gpa = gpa;
}

}

// Usage — all three constructors:
Student s1 = new Student();
Student s2 = new Student("Alice");
Student s3 = new Student("Bob", 20, 3.8);

```

this() — Constructor Chaining

Calling `this(...)` as the FIRST statement in a constructor delegates to another overloaded constructor. This eliminates code duplication — only the canonical (full) constructor contains the actual assignment logic. All others funnel through it.

6.4 Access Modifiers

Access modifiers control which code can see a class, field, or method. They are the primary mechanism of encapsulation in Java.

Modifier	Same Class	Same Package	Subclass	Anywhere
<code>private</code>	✓ Yes	✗ No	✗ No	✗ No
<code>(package-private)</code>	✓ Yes	✓ Yes	✗ No	✗ No
<code>protected</code>	✓ Yes	✓ Yes	✓ Yes	✗ No
<code>public</code>	✓ Yes	✓ Yes	✓ Yes	✓ Yes

Practical Guidelines

- **Fields:** Always private. Never make fields public — it breaks encapsulation and makes validation impossible.
- **Methods:** public if part of the class's contract; private if implementation detail.

- **Constructors:** Usually public. private constructors are used in Singleton and Factory patterns.
- **Constants:** public static final is acceptable and conventional: public static final double TAX = 0.15;

6.5 The this Keyword & Instance References

Inside an instance method or constructor, `this` is a reference to the current object — the object on which the method was invoked. It has three primary uses:

```
// Use 1: disambiguate field vs parameter with the same name
public Person(String name, int age) {
    this.name = name; // this.name = field; name = parameter
    this.age = age;
}

// Use 2: constructor chaining (call another constructor)
public Person(String name) {
    this(name, 0); // must be FIRST statement
}

// Use 3: pass current object as argument to another method
public void register(Registry r) {
    r.add(this); // pass self to the registry
}

// Use 4: method chaining — return this for fluent API
public Person setName(String name) {
    this.name = name; return this;
}
public Person setAge(int age) {
    this.age = age; return this;
}

// Fluent chaining:
Person p = new Person().setName("Alice").setAge(25);
```

6.6 Getters & Setters — Enforcing Encapsulation

Getters (accessors) and setters (mutators) are public methods that provide controlled access to private fields. The key advantage: setters can validate input before modifying the field, preventing objects from reaching an invalid state.

```
public class BankAccount {
    private String owner;
    private double balance;
    private String accountNumber;

    public BankAccount(String owner, String accountNumber, double
initialBalance) {
        this.owner = owner;
        this.accountNumber = accountNumber;
    }
}
```

```

        setBalance(initialBalance); // reuse validation in constructor
    }

    // — GETTER — read-only access —
    public double getBalance() { return balance; }
    public String getOwner() { return owner; }
    public String getAccountNumber() { return accountNumber; }

    // — SETTER — with validation —
    public void setBalance(double balance) {
        if (balance < 0) {
            throw new IllegalArgumentException("Balance cannot be
negative");
        }
        this.balance = balance;
    }

    public void setOwner(String owner) {
        if (owner == null || owner.isBlank())
            throw new IllegalArgumentException("Owner name cannot be
blank");
        this.owner = owner.trim();
    }

    // — BUSINESS METHODS —
    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Deposit must
be positive");
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Amount
must be positive");
        if (amount > balance) throw new
IllegalArgumentException("Insufficient funds");
        balance -= amount;
    }

    @Override
    public String toString() {
        return String.format("Account[%s, owner=%s, balance=$%.2f]",
accountNumber, owner, balance);
    }
}

```

6.7 Creating & Using Objects — References vs Primitives

Understanding the difference between primitive values and object references is essential to predicting how Java programs behave, especially when copying variables or passing them to methods.

Memory Model

```
// Primitive: value lives directly in the variable's memory slot
```

```

int x = 42;
int y = x;      // y gets a COPY of 42
y = 99;         // x is still 42 — independent copies

// Object: variable holds a REFERENCE (address), object is on the heap
BankAccount a1 = new BankAccount("Alice", "ACC001", 1000);
BankAccount a2 = a1;    // a2 gets a COPY of the REFERENCE
                        // BOTH a1 and a2 point to the SAME object!
a2.deposit(500);        // a1.getBalance() is now 1500 too!

// null — a reference that points to no object
BankAccount a3 = null;
a3.deposit(100); // NullPointerException! — always check for null first

// Proper null check:
if (a3 != null) a3.deposit(100);

```

The Heap vs The Stack

Primitive local variables and object references live on the call stack (fast, small). Object data lives on the heap (large, garbage-collected). When a method returns, its stack frame is destroyed — heap objects persist until no references point to them.

6.8 toString(), equals() & Object Utility Methods

Every class in Java implicitly extends `Object`, which provides base implementations of `toString()` and `equals()`. Overriding these is considered standard practice for any class representing domain data.

```

public class Point {
    private final double x, y;

    public Point(double x, double y) { this.x=x; this.y=y; }

    @Override
    public String toString() {
        return String.format("Point(%.2f, %.2f)", x, y);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true; // same object
        if (!(obj instanceof Point p)) return false; // pattern match (Java
16+)
        return Double.compare(x, p.x) == 0 && Double.compare(y, p.y) == 0;
    }

    @Override
    public int hashCode() {
        return java.util.Objects.hash(x, y); // always override hashCode
with equals!
    }
}

```

 The equals/hashCode Contract

If you override equals(), you MUST also override hashCode(). Java's HashMap, HashSet, and other hash-based collections use hashCode() for bucketing — if two equal objects have different hash codes, those collections break in subtle, hard-to-debug ways.

 Lab 6 — Classes, Encapsulation & OOP Design **Exercise Set A: Class Fundamentals**

- Exercise 24. Rectangle Class:** Design a Rectangle class with private fields length and width. Include a parameterised constructor, getters/setters with validation (no negative dimensions), and methods: area(), perimeter(), isSquare(), scale(double factor) — returns a new Rectangle.
- Exercise 25. Circle Class:** Design a Circle with a private radius field. Include constructor, validated setter, and methods: area(), circumference(), isUnitCircle() (radius==1), contains(double x, double y) — returns true if point is inside the circle.
- Exercise 26. Time Class:** Design a Time class representing HH:MM:SS. Validate hours (0–23), minutes (0–59), seconds (0–59). Implement addSeconds(int s) — advances time by s seconds, wrapping correctly. Include toString() returning 'HH:MM:SS'.
- Exercise 27. Fraction Class:** Design a Fraction class with numerator and denominator fields. Automatically reduce to lowest terms using GCD in the constructor. Implement add(), subtract(), multiply(), divide() — each returning a new Fraction. Include equals() and toString() ('3/4').
- Exercise 28. Temperature Class:** Design a Temperature class that stores a value and unit ('C','F','K'). Implement toCelsius(), toFahrenheit(), toKelvin() methods each returning a new Temperature. Validate that Kelvin values cannot be negative.

 Exercise Set B: BankAccount System (Core Project)

- Exercise 29. BankAccount Foundation:** Implement the BankAccount class from Section 6.6 in full. Add: a transactionHistory ArrayList<String>, where each deposit/withdrawal appends a record like 'Deposit: +\$200.00 | Balance: \$1200.00'. Add printStatement() method.
- Exercise 30.** Add a static field nextAccountNumber (starts at 1000) that auto-increments each time a BankAccount is created. The constructor should assign this automatically — callers only provide owner and initial balance.
- Exercise 31. SavingsAccount Extension:** Create a SavingsAccount subclass that adds an interestRate field and an applyMonthlyInterest() method. Add a minimum balance rule — withdrawals are rejected if they would drop the balance below \$100.
- Exercise 32.** Write a BankSimulation class with a main() that creates 3 accounts, performs a sequence of deposits and withdrawals (including at least 2 that should fail validation), and prints the final statement for each account.
- Exercise 33.** Add a transfer(BankAccount target, double amount) method to BankAccount that atomically debits from this and credits to target. Both the source and target transaction histories must be updated.
- Exercise 34. Account Comparison:** Override equals() and hashCode() in BankAccount based on accountNumber. Override toString() with a multi-line formatted account summary. Demonstrate equals() correctness with at least 3 test cases.



Exercise Set C: Student Record System

- Exercise 35. **Student Class:** Design a Student with fields: studentId (auto-assigned), name, email, and an ArrayList<Double> grades. Include methods: addGrade(double), getAverage(), getHighestGrade(), getLowestGrade(), getLetterGrade(), toString().
- Exercise 36. **Course Class:** Design a Course with fields: courseCode, courseName, instructor, and ArrayList<Student> roster. Include: enroll(Student), drop(Student), printRoster(), getClassAverage(), getTopStudent() returning the Student with the highest average.
- Exercise 37. Write a Classroom simulation: create 1 course and 5–8 students. Enroll them, add 4 grades each, print the full roster report, identify the top and bottom students, and show the class average.
- Exercise 38. **Constructor Chaining Practice:** Create an Employee class with fields: id (auto), name, department, salary, and hireDate (use a String). Provide 3 overloaded constructors (name-only, name+department, full). All must chain to the canonical constructor.



Exercise Set D: Challenge & Design Problems

- Exercise 39. **Immutable Class:** Design an immutable Money class (all fields final, no setters). Include: amount (BigDecimal) and currency (String). Implement add(Money), subtract(Money), multiply(double), and compareTo(Money). Throw exceptions for currency mismatch.
- Exercise 40. **Builder Pattern:** Implement a Pizza class with many optional fields: size, crust, sauce, and a List<String> toppings. Build a static inner Pizza.Builder class with fluent setter methods ending in build() that returns the Pizza. Demonstrate building 3 different pizzas.
- Exercise 41. **Singleton Pattern:** Implement a Configuration class that can only have one instance (Singleton). It stores key-value settings in a HashMap. Provide static getInstance(), set(key, value), get(key), and reset() methods. Demonstrate that multiple calls to getInstance() return the same object.
- Exercise 42. **Object Array Operations:** Create an array of 10 Student objects. Write methods to: sort by name (alphabetically), sort by average (descending), find a student by name (linear search), and print a ranked leaderboard.